

Algorithms for `BSTBag`

- Student ID: 3118197
 - Name: WEI-TSUNG CHENG
-

1. Algorithm for `add(element)`

Purpose

Add an element `x` into the bag.

- If `x` is already present in the bag, increase its count. Otherwise, create a new node in the BST with count 1.

Input

- An element `x`.

Output

- None

Algorithm

1. If `x` is null, do nothing and return.
 2. Create a counted element `new_x` with `new_x`'s value is `x`
`new_x`'s count = 1
 3. If the tree is empty
 - 3.1. Create a new node `newNode` storing `new_x`, with empty left and right children.
 - 3.2. Set `newNode` as the root of the tree.
 - 3.3. Increase the bag size by 1.
 - 3.4. Stop.
 4. Otherwise (the tree is not empty)
 - 4.1. Let `current` point to the root.
 - 4.2. Let `parent` be null.
 - 4.3. While `current` is not null, repeat:
 - a. Set `parent` to `current`.
 - b. Compare `new_x` with the counted element stored at `current` (comparing element values only):
 - If they are equal:
 - i. Increase the count stored at `current` by 1.
 - ii. Increase the bag's size by 1.
 - iii. Stop.
 - If the element value in `new_x` is less than the element value at `current`:
 - i. Move `current` to its left child.
 - Otherwise (the element value in `new_x` is greater than the element value at `current`):
 - i. Move `current` to its right child.
 5. When the loop finishes, `current` is null and `parent` is the node under which the new node must be attached.
 - 5.1. Create a new node `newNode` storing `new_x`
 - 5.2. If the element value in `new_x` is less than the element value at `parent`, attach `newNode` as the left child of `parent`; otherwise, attach `newNode` as the right child of `parent`.
 - 5.3. Increase the bag size by 1.
-

2. Algorithms for `remove(element)`

Removal is described in four parts:

- The top-level `remove(x)`.
 - The recursive procedure `recursiveRemove(node, target)` that removes one occurrence from a subtree.
 - The helper `findMin(node)` that finds the minimum node in a subtree.
 - The helper `deleteMin(node)` that deletes the minimum node in a subtree.
-

2.1 Top-level `remove(x)`

Purpose

Remove exactly one occurrence of an element `x` from the bag.

- If `x` appears multiple times, only its count is decreased by 1.
- If this is the last occurrence, the node is removed.

Input

- An element `x`.

Output

- None

Effect

- If `x` is present in the bag, the bag size decreases by 1.
- If `x` is not present, the bag remains unchanged.

Algorithm

1. If the tree is empty, or if `x` is null, return.
 2. Create a temporary counted element `target` with
 - `target`'s element is `x`
 - `target`'s count is 1
 3. Set the root of the tree to the result of
 - `recursiveRemove(root, target)`.
-

2.2 `recursiveRemove(node, target)`

Purpose

Remove one occurrence of the element specified by `target` from the subtree rooted at `node`.

Input

- `node`: root of the current subtree (possibly null).
- `target`: a counted element whose element value is the target to remove.

Output

- The new root of this subtree after removal.

Algorithm

1. If `node` is null, return null.
2. Compare the element value of `target` with the element stored at `node`.
 - 2.1. If the value of `target` is less than the element at `node`
 - a. Set `node.left` to `recursiveRemove(node.left, target)`.
 - b. Return `node`.
 - 2.2. Else if the value of `target` is greater than the element at `node`
 - a. Set `node.right` to `recursiveRemove(node.right, target)`.
 - b. Return `node`.
 - 2.3. Otherwise (the value of `target` is equal to the element at `node`):
 - a. Let `c` be the count stored in the counted element at `node`.
 - b. If `c` is greater than 1
 - i. Decrease the count at `node` by 1.
 - ii. Decrease the bag size by 1.
 - iii. Return `node`.
(The tree structure does not change.)
 - c. Otherwise (`c` is equal to 1; this is the last occurrence):
 - i. Decrease the bag size by 1.
 - ii. If `node` has no left child and no right child
 - Return null. (The leaf node is deleted.)
 - iii. If `node` has no left child but has a right child
 - Return `node.right`. (The right child becomes the new root of this subtree.)
 - iv. If `node` has no right child but has a left child
 - Return `node.left`. (The left child becomes the new root of this subtree.)
 - v. Otherwise, `node` has both a left and a right child:
 - Let `s` be the result of `findMin(node.right)`
(the node with the smallest element value in the right subtree).
 - Copy the element value and the count from `s` into the counted element stored at `node`.

- Remove the minimum node from the right subtree
 - Set `node.right` to `deleteMin(node.right)`.
 - Return `node`.
-

2.3 Helper `findMin(node)`

Purpose

Return the node with the smallest element value in the subtree rooted at `node`.

Input

- `node` : root of the current subtree.

Output

- The node that contains the smallest element value in this subtree, or `null` if the subtree is empty.

Algorithm

1. If `node` is null, return null.
 2. While `node.left` is not null, repeat:
 - 2.1. Move `node` to its left child.
 3. Return `node`.
-

2.4 Helper `deleteMin(node)`

Purpose

Delete the minimum element from the subtree rooted at `node`.

- This procedure only changes the shape of the tree.

Input

- `node` : root of the current subtree.

Output

- The new root of this subtree after deleting the minimum node.

Algorithm

1. If `node` is null, return null.
2. If `node.left` is null
 - 2.1. Return `node.right`.
3. Otherwise
 - 3.1. Set `node.left` to `deleteMin(node.left)`.
 - 3.2. Return `node`.

3. Algorithm for the iterator `BSTIterator()`

The iterator performs an in-order traversal of the BST and, for each node, returns its element as many times as indicated by its count.

3.1 Iterator state

`BSTIterator` object maintains the following internal state:

- A stack `stack` of tree nodes, used to simulate recursive in-order traversal.
 - An integer `repeatCount`, the number of additional times the current element must still be returned.
 - A variable `currentElement`, storing the counted element at the current node.
-

3.2 Constructor `BSTIterator()`

Purpose

Initialise the iterator so that it is ready to return the first element in order (in-order).

Input

- None

Output

- None

Algorithm

1. Set `repeatCount` to 0.
 2. Set `currentElement` to null.
 3. Make `stack` empty.
 4. Call `pushLeft(root)` where `root` is the root of the BST.
-

3.3 Helper `pushLeft(node)`

Purpose

Push all nodes on the path from `node` down to the leftmost node onto the stack.
This prepares the stack so that the next popped node is the smallest (in-order) node in the given subtree.

Input

- `node` : the starting node of a subtree.

Output

- None

Algorithm

1. While `node` is not null, repeat:
 - 1.1. Push `node` onto `stack`.
 - 1.2. Move `node` to its left child.
-

3.4 Method `hasNext()`

Purpose

Determine whether the iterator has another element to return.

Input

- None

Output

- A boolean value indicating whether there is a next element.

Algorithm

1. If `repeatCount` is greater than 0
 - 1.1. Return `true`. (There are still remaining copies of the current element.)
 2. Else if `stack` is not empty
 - 2.1. Return `true`. (There are more BST nodes to visit.)
 3. Otherwise
 - 3.1. Return `false`.
-

3.5 Method `next()`

Purpose

Return the next element in the bag

Input

- None

Output

- The next element.

Algorithm

1. If `hasNext()` is `false`
 - 1.1. There is no next element.
2. If `repeatCount` is greater than 0
 - 2.1. Decrease `repeatCount` by 1.
 - 2.2. Return `currentElement`'s element value.

3. Otherwise (`repeatCount` is equal to 0)

- 3.1. Pop the top node `n` from `stack` .
- 3.2. Set `currentElement` to the counted element stored at node `n` .
- 3.3. Set `repeatCount` to `currentElement` 's count minus 1.
- 3.4. If `n` has a right child
 - Call `pushLeft(n.right)` to push all nodes on the leftmost path of the right subtree onto `stack` .
- 3.5. Return `currentElement` 's element value.